

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint Based Problem Solving

Course Code:	ITA0517	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Yoga Madha A	Reg. No.:	192425058
Section / Batch:	B	Date:	24-08-2026

Code of Conduct

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Gesture Recognition System

1. Problem Statement

The objective is to design a **real-time Gesture Recognition System** using a webcam and OpenCV. The system should capture the user's hand gestures, identify simple gestures, and provide an immediate response.

The system should mainly perform three functions:

- **Capture:** Obtain hand gestures from a webcam.
- **Identify:** Recognize simple hand gestures.
- **Respond:** Perform or display the corresponding action in real time.

Examples of gestures include **Fist, Open Palm, Pointing, Victory, and Thumbs Up**.

2. Objectives

The main objectives of the system are:

1. To capture live video from a webcam.
2. To detect the user's hand from each frame.
3. To remove unnecessary background information.
4. To extract meaningful hand features.
5. To classify different hand gestures.
6. To provide a fast real-time response.
7. To develop a simple and computationally efficient system.

3. Input and Output

Input

The input is a **live video stream from a webcam**.

The webcam continuously captures frames containing the user's hand.

Output

The output is the **recognized gesture and its corresponding action**.

For example:

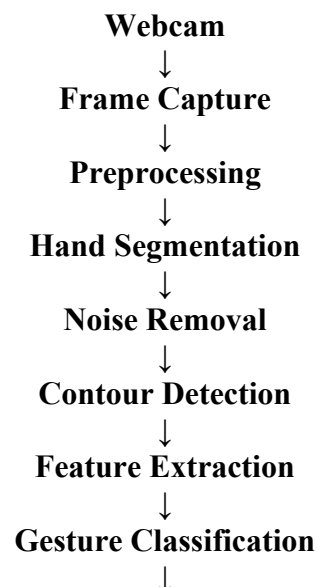
Open Palm → START

Fist → STOP

Pointing → SELECT

Victory → NEXT

4. Proposed OpenCV Pipeline



Real-Time Response

5. Webcam Frame Capture

OpenCV is used to access the webcam using the VideoCapture() function.

The camera continuously captures frames.

For example:

Frame 1 → Frame 2 → Frame 3 → Frame 4 → ...

Each frame is processed separately.

Importance

Webcam capture allows the system to recognize gestures dynamically instead of using a single static image.

6. Image Preprocessing

The captured image may contain noise, shadows and unnecessary details.

Therefore, preprocessing is performed before detecting the hand.

The main preprocessing operations are:

- Image resizing.
- Gaussian filtering.
- Color-space conversion.
- Thresholding.

Color-space conversion

The BGR image can be converted into **HSV** or **YCrCb**.

HSV is useful because it separates color information from brightness, making segmentation easier under moderate lighting variations.

7. Hand Segmentation

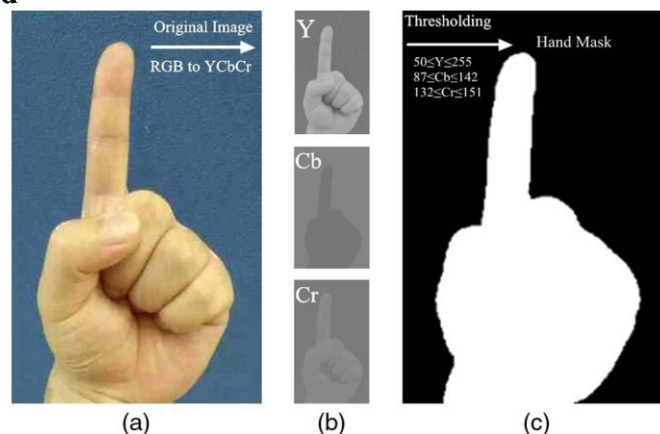
After preprocessing, the hand must be separated from the background.

Thresholding is applied to create a **binary image**.

In the binary image:

White → Possible hand region

Black → Background



The quality of segmentation directly affects the accuracy of the final gesture recognition.

8. Noise Removal

The segmented image may contain small unwanted regions.

Morphological operations can be used to improve the binary mask.

Erosion

Removes small unwanted regions.

Dilation

Expands the detected hand region.

Opening

Helps remove small noise.

Closing

Helps fill small gaps in the detected region.

This produces a cleaner hand mask for contour detection.

9. Contour Detection

After segmentation, OpenCV is used to detect the hand boundary.

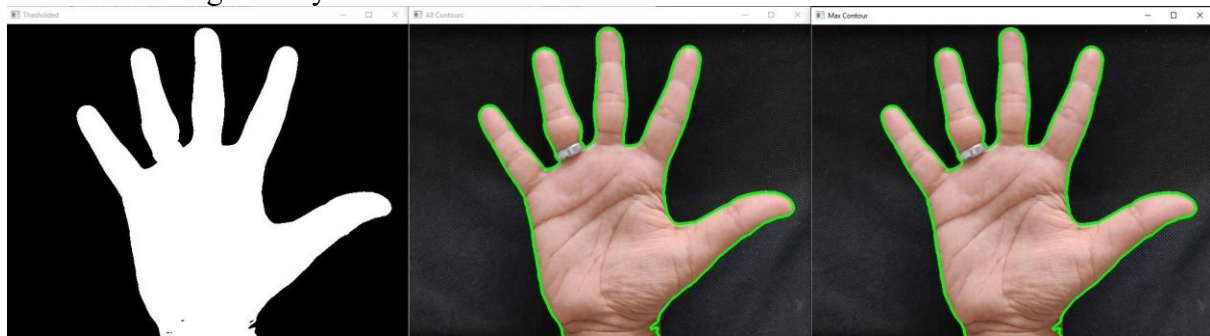
The findContours() function identifies the contours present in the binary image.

The largest suitable contour is generally selected as the hand.

Why contours are important

A contour provides information about:

- Hand boundary.
- Hand shape.
- Hand area.
- Finger positions.
- Overall geometry.



10. Feature Extraction

Feature extraction is an important stage of the system.

Instead of analyzing every pixel, useful characteristics of the hand are extracted.

The main features are:

1. Contour

Represents the boundary of the hand.

2. Convex Hull

Represents the outer boundary of the hand.

3. Convexity Defects

Represents gaps between fingers.

4. Finger Count

Estimates the number of extended fingers.

5. Hand Area

Represents the size of the detected hand region.

6. Aspect Ratio

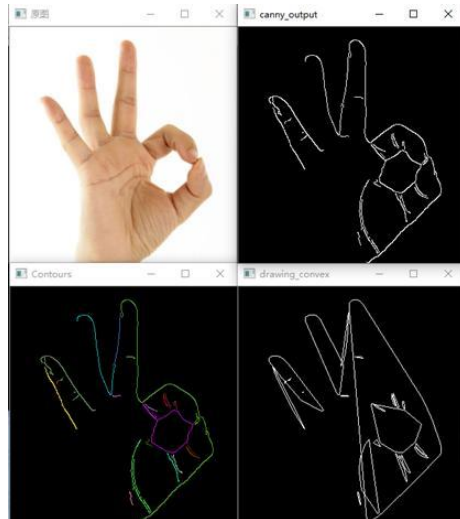
Represents the relationship between hand width and height.

7. Finger Angles

Can be used to identify specific finger positions.

11. Convex Hull

A **convex hull** is the smallest convex boundary surrounding the hand contour.



It helps identify the outermost points of the hand, such as fingertips and the sides of the hand.

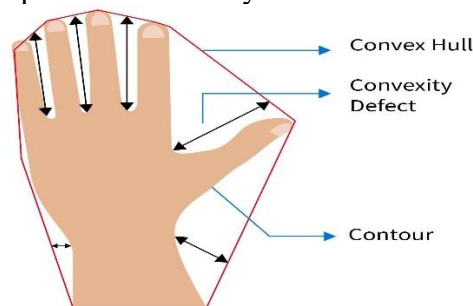
Justification

The convex hull simplifies the hand shape and makes it easier to analyze the position of fingers.

12. Convexity Defects

Convexity defects are the regions between the actual contour and its convex hull.

The spaces between fingers can produce convexity defects



SCALER

These defects are useful for estimating the number of extended fingers.

For example:

Open Palm → Multiple finger gaps

Fist → Very few or no finger gaps

Justification

Convexity defects provide important geometric information for distinguishing simple hand gestures.

13. Finger Count

Finger count is one of the simplest features for gesture recognition.

The system analyzes the convexity defects and fingertip positions to estimate the number of extended fingers.

Example:

- **0 fingers** → **Fist**
- **1 finger** → **Pointing**
- **2 fingers** → **Victory**
- **5 fingers** → **Open Palm**

Finger count is fast to calculate and works well for basic gestures.

14. Gesture Classification

After extracting the features, the system classifies the gesture.

For this problem, a **rule-based classifier** is suitable because only simple gestures are required.

Example Rules

If finger count = 0

→ Gesture = **Fist**

If finger count = 1

→ Gesture = **Pointing**

If finger count = 2

→ Gesture = **Victory**

If finger count = 5

→ Gesture = **Open Palm**

A separate rule can be added for **Thumbs Up** based on thumb orientation and other geometric features.

15. Alternative Classification Methods

For more advanced gesture recognition, machine-learning methods can be used.

KNN

K-Nearest Neighbors compares the extracted features with previously stored examples.

SVM

Support Vector Machine creates decision boundaries between different gesture classes.

Neural Network

A neural network can learn relationships between hand features and gesture categories.

CNN

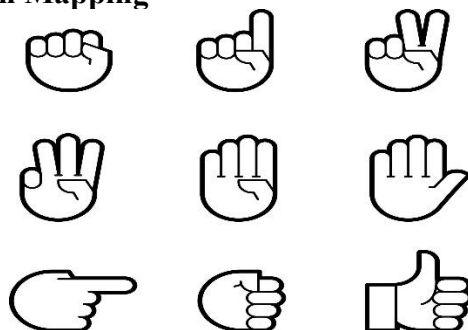
A Convolutional Neural Network can learn image features directly from hand images.

Selected Method

For this system, **rule-based classification** is preferred because it:

- Is simple.
- Is fast.
- Requires no large training dataset.
- Has low computational requirements.
- Is suitable for real-time processing.

16. Gesture-to-Action Mapping



Gesture	Detection	Action
Fist	0 fingers	STOP
Pointing	1 finger	SELECT
Victory	2 fingers	NEXT
Open Palm	5 fingers	START

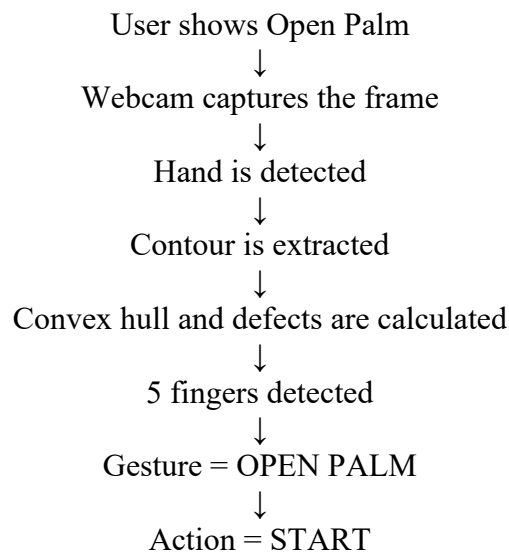
Gesture	Detection	Action
Thumbs Up	Thumb extended	ACCEPT

This mapping allows the gesture recognition system to work as an interactive control system.

17. Real-Time Response

After classification, the system immediately displays the recognized gesture on the webcam screen.

For example:



The same process is continuously repeated for every new webcam frame.

18. Constraint Analysis

Constraint 1: Real-Time Processing

The system must provide a response without noticeable delay.

Solution: Use lightweight OpenCV operations and a rule-based classifier.

Constraint 2: Lighting

Different lighting conditions can affect hand segmentation.

Solution: Use HSV/YCrCb color space, smoothing and appropriate thresholding.

Constraint 3: Background

A complex background can produce false contours.

Solution: Use a simple background or define a Region of Interest (ROI).

Constraint 4: Hand Orientation

The hand can appear at different positions and angles.

Solution: Use contour geometry, convex hull, finger positions and angles.

Constraint 5: Recognition Accuracy

Incorrect finger detection can result in an incorrect gesture.

Solution: Combine multiple features rather than depending only on finger count.

Constraint 6: Processing Resources

The system should run on a normal computer.

Solution: Use OpenCV and lightweight classification instead of a computationally expensive deep-learning model.

19. Advantages

The proposed system provides the following advantages:

1. Works with a normal webcam.
2. Provides real-time gesture recognition.
3. Easy to implement using OpenCV.
4. Requires low computational resources.
5. Does not require a large dataset for basic gestures.
6. Can recognize multiple simple gestures.
7. Can be integrated with other applications.
8. Provides touch-free human-computer interaction.

20. Applications

The gesture recognition system can be used in:

Smart Home

Gestures can control lights, fans and other devices.

Presentation Control

Gestures can be used to move between presentation slides.

Gaming

Hand gestures can act as game controls.

Healthcare

Touch-free interfaces can be useful in specific healthcare environments.

Human-Computer Interaction

Users can interact with computers without directly using a keyboard or mouse.

Accessibility

Gesture-based interfaces can provide alternative interaction methods.

21. Performance Evaluation

The system can be evaluated using:

Accuracy

Measures how many gestures are correctly identified.

Response Time

Measures the time taken from gesture appearance to recognition.

Frame Rate

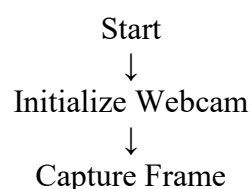
Measures how many frames can be processed per second.

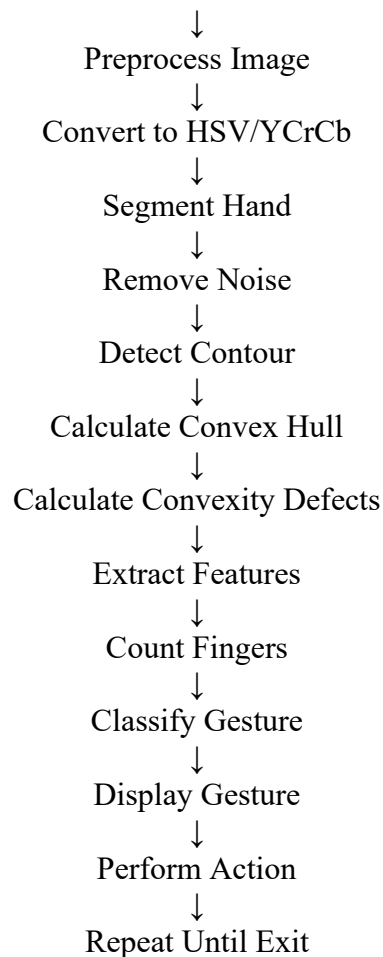
Robustness

Tests whether the system works under:

- Different lighting.
- Different backgrounds.
- Different hand positions.
- Different hand orientations.

22. Overall Flow





23. Justification of Feature Extraction

Contour is selected because it represents the boundary and shape of the hand.

Convex Hull is selected because it provides the outer structure of the hand and helps identify fingertips.

Convexity Defects are selected because they represent gaps between fingers.

Finger Count is selected because it provides a simple and effective way to distinguish basic gestures.

Area and Aspect Ratio are selected to identify the hand region and reduce false detections. Therefore, these features provide sufficient information for recognizing simple gestures while maintaining fast processing.

24. Justification of Classification

A **rule-based classifier** is the most suitable method for this particular problem because the required gestures are simple and predefined.

It provides:

- Fast classification.
- Low computational cost.
- Easy implementation.
- No requirement for a large training dataset.
- Easy modification of gesture rules.
- Suitable real-time performance.

For a larger gesture vocabulary, **SVM, KNN or CNN** can be introduced.

25. Conclusion

The proposed **Gesture Recognition System** uses an OpenCV-based pipeline consisting of **webcam capture, preprocessing, hand segmentation, contour detection, feature extraction, gesture classification and real-time response**.

The use of **contours, convex hull, convexity defects and finger count** provides effective features for identifying simple hand gestures. A **rule-based classifier** is suitable because it is fast, lightweight and easy to implement.

Therefore, the proposed system satisfies all the required constraints of **capturing hand gestures, identifying simple gestures and responding in real time**, making it suitable for human-computer interaction, smart-home control, presentation control, gaming and other applications.